

# REFCHECKER: Reference-based Fine-grained Hallucination Checker and Benchmark for Large Language Models

Xiangkun Hu<sup>1</sup>, Dongyu Ru<sup>1</sup>, Lin Qiu<sup>1</sup>, Qipeng Guo<sup>2</sup>, Tianhang Zhang<sup>1</sup>  
Yang Xu<sup>3</sup>, Yun Luo<sup>4</sup>, Pengfei Liu<sup>3</sup>, Yue Zhang<sup>4</sup> and Zheng Zhang<sup>1</sup>

<sup>1</sup>Amazon AWS AI <sup>2</sup>Shanghai AI Lab

<sup>3</sup>Shanghai Jiaotong University <sup>4</sup>Westlake University

{xiangkhu, rudongyu, qulin, zzthang, zhaz}@amazon.com, guoqipeng@pjlab.org.cn  
{xuyang0112, pengfei}@sjtu.edu.cn, {luoyun, yue.zhang}@wias.org.cn

## Abstract

Large Language Models (LLMs) have shown impressive capabilities but also a concerning tendency to hallucinate. This paper presents REFCHECKER, a framework that introduces *claim-triplets* to represent claims in LLM responses, aiming to detect fine-grained hallucinations. In REFCHECKER, an extractor generates claim-triplets from a response, which are then evaluated by a checker against a reference. We delineate three task settings: Zero, Noisy and Accurate Context, to reflect various real-world use cases. We curated a benchmark spanning various NLP tasks and annotated 11k claim-triplets from 2.1k responses by seven LLMs. REFCHECKER supports both proprietary and open-source models as the extractor and checker. Experiments demonstrate that claim-triplets enable superior hallucination detection, compared to other granularities such as response, sentence and sub-sentence level claims. REFCHECKER outperforms prior methods by 6.8 to 26.1 points on our benchmark and the checking results of REFCHECKER are strongly aligned with human judgments<sup>1</sup>.

## 1 Introduction

Large Language Models (LLMs) have sparked a revolution in Natural Language Processing (NLP), covering diverse tasks with a unified architecture (Zhao et al., 2023). However, LLMs exhibit a tendency to generate hallucinated contents that can be difficult to discern, posing a potential risk of misleading users. (Huang et al., 2023). Hallucination detection is therefore an important task (Manakul et al., 2023; Min et al., 2023; Chern et al., 2023).

Detecting hallucination is essentially a job of comparing a response against a reference. However, several challenges remain: determining the appropriate unit of analysis for comparison, building a comprehensive benchmark reflecting real-

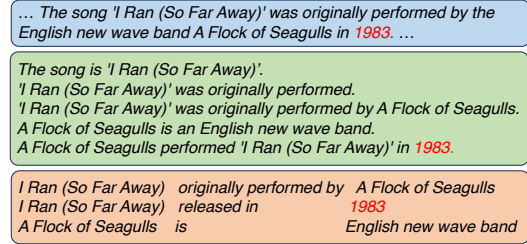


Figure 1: An example response split into **sentence**, **sub-sentence** (Min et al., 2023), **triplets**, and the hallucination **1983**. Triplets define the boundary of claims more clearly, are fine-grained and covers non-overlapping facts (unlike sub-sentences).

world LLM applications, developing a unified, automated framework that scales detection across diverse tasks. In this work, we introduce REFCHECKER to tackle these challenges.

In terms of checking granularity, response level checking (Lin et al., 2022; Li et al., 2023) suffices if the query and response is about a simple fact. However, when responses are complex and long, response-level checking is not only uninformative but can also cause false-negative when hallucination is local. This is common in real-world use cases, for example, the response from Llama 2 (Touvron et al., 2023) in our benchmark (described later) contains 150 tokens on average. For fine-grained detection, Manakul et al. (2023) takes the sentences in a response as the checking units. Min et al. (2023) and Chern et al. (2023) further extracts short phrases (we term them as sub-sentences) as the claims, as one sentence may contain multiple hallucinations, or one hallucination may span across sentence boundaries. However, sub-sentences are structurally difficult to define, making it challenging to form high-quality demonstrations to be used by LLMs with in-context learning. To this end, we propose to extract knowledge triplets as checking units. We show an example with different granularity in Figure 1. Triplets exhibit fine-grained and clearly separated semantics. These triplets are called *claim-triplets*. Experi-

<sup>1</sup>This work is open sourced at <https://github.com/amazon-science/RefChecker>

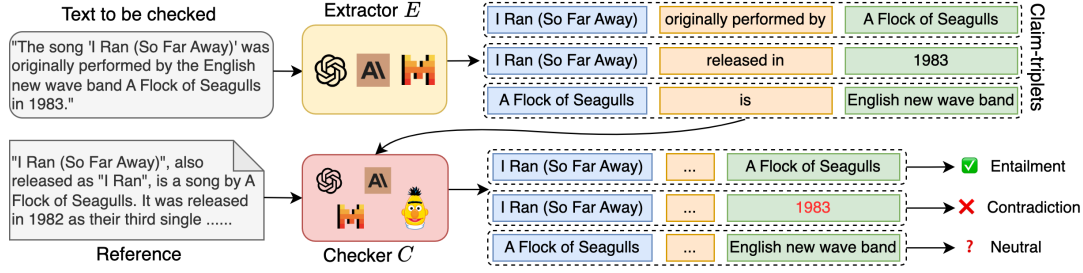


Figure 2: The REFCHECKER framework comprises two main components: an extractor denoted as  $E$  and a checker denoted as  $C$ . Given a text to be checked, typically a response generated by an LLM, the extractor takes it as input and generates a set of knowledge triplets, referred to as claim-triplets. Subsequently, the checker assesses each claim-triplet by comparing it against a reference, assigning a hallucination label based on the evaluation.

ments show that checking with claim-triplets gains 4 to 9 points of improvement over other granularity on our benchmark (cf. Sec. 5.1).

For better evaluation, we curate a comprehensive dataset on which we can benchmark hallucination under different context quality and availability. Using this benchmark, we conducted human evaluation on 2,100 responses from 7 LLMs. We annotated 11k claim-triplets with 95% Inter-Annotator Agreement on 23% of the annotations. Compared with recent proposed benchmarks (Manakul et al., 2023; Min et al., 2023; Chern et al., 2023), it covers a more diverse range of domains and tasks, with more LLMs and responses evaluated (see Table 1). As expected, we found by human evaluation that hallucination is the most pronounced (cf. Appendix A.4) when LLMs are asked to generate responses solely from its memory (Zero Context), followed by responding to noisy references in RAG (retrieval augmented generation) setting (Shuster et al., 2021) (Noisy Context) and finally when references are more or less noisy free (Accurate Context).

REFCHECKER (Figure 2) is a fully automated framework that scales hallucination detection across different tasks. The extractor generates claim-triplets from the response and the checker evaluates each of the claim-triplets by comparing them with the reference. In contrast to recent work that only differentiates factual and non-factual claims, the checker in REFCHECKER also considers unverifiable claims when the reference is insufficient for checking. Both the extractor and checker supports proprietary (e.g. GPT-4 (OpenAI et al., 2023) or Claude 2 (Anthropic, 2023)) and open-source models (e.g. Mistral (Jiang et al., 2023) and RoBERTa (Liu et al., 2019) based models). We made careful study to select and recommend configurations that give results consistent with human

annotation, and show 6.8 to 26.1 points of improvement over the best alternative (Sec. 5.2).

Our key contributions include:

- **Claim-triplet formulation:** Our novel “claim-triplet” analysis outperforms existing methods by up to 9 points, pinpointing factual inconsistencies within responses.
- **Comprehensive benchmark:** We developed a robust benchmark covering three classes of real-world LLM tasks with 11,000 manually annotated claim-triplets across 7 LLMs.
- **Automatic checking framework:** Our REFCHECKER framework extracts and verifies claim-triplets, boosting consistency by 19-26 points over prior methods and works with both proprietary and open-source models.

## 2 Related Work

We undertake a review of prior work relevant to our study and compare them with REFCHECKER. The comparative analysis with three representative methods is encapsulated in Table 1.

**Hallucinations in LLMs** Hallucinations frequently occur in NLP tasks like summarization (Maynez et al., 2020; Cao et al., 2022), machine translation (Guerreiro et al., 2023a,b), dialog systems (Honovich et al., 2021; Dziri et al., 2022) and RAG (Shuster et al., 2021). According to a recent comprehensive survey (Huang et al., 2023), hallucinations can be categorized to factuality hallucinations and faithfulness hallucinations. Factuality hallucinations involve claims contradicted by real-world facts, while faithfulness hallucinations are inconsistent with the input content. Recent research on hallucination detection primarily concentrates on factuality hallucinations, such as SelfCheck-GPT (Manakul et al., 2023), FActScore (Min et al.,

| Method       | Context Setting                                   | Claim Extraction        |                                       | Checking |                                               | Benchmark             |                                           |                     |
|--------------|---------------------------------------------------|-------------------------|---------------------------------------|----------|-----------------------------------------------|-----------------------|-------------------------------------------|---------------------|
|              |                                                   | Claim                   | Extractor                             | Label    | Checker                                       | Domain                | Task                                      | Evaluated Responses |
| SelfCheckGPT | Zero Context                                      | Sentence                | -                                     | 3-way    | GPT                                           | Wikipedia             | Bio Generation                            | 238 from GPT-3      |
| FActScore    | Zero Context                                      | Sub-sentence            | GPT                                   | Binary   | GPT                                           | Wikipedia             | Bio Generation                            | 505 from 3 LLMs     |
| FacTool      | Zero Context                                      | Sub-sentence            | GPT                                   | Binary   | GPT                                           | Wikipedia             | QA                                        | 514 from ChatGPT    |
|              |                                                   | Snippet Statement Tuple |                                       |          |                                               | Python Math Sci. Text | Code Generation Math Problems Sci. Review |                     |
| REFCHECKER   | Zero Context<br>Noisy Context<br>Accurate Context | Triplet                 | GPT<br>Claude<br>Mixtral*<br>Mistral* | 3-way    | GPT<br>Claude<br>AlignScore*<br>NLI*<br>RepC* | Wikipedia<br>Web      | QA<br>RAG<br>Summarization<br>IE          | 2,100 from 7 LLMs   |

Table 1: A comparison of REFCHECKER with previous approaches for hallucination detection. The “\*” symbols alongside the extractors and checkers indicate that these models are open-sourced. REFCHECKER uses triplets as the claims instead of sentences or sub-sentences. The REFCHECKER benchmark covers more context settings and more diverse tasks. The human evaluation covers more LLMs and responses. REFCHECKER pipeline supports both proprietary and open-source models, facilitating broader adoption across various applications.

2023) and FacTool (Chern et al., 2023). We address both factuality and faithfulness hallucinations and further categorizing them into three contextual settings to align with real-world use cases.

**Granularity of Claims** Claims are pivotal for evaluating responses generated by LLMs. Response level checking (Lin et al., 2022; Li et al., 2023) is too coarse-grained for long-form responses. For fine-grained detection, sentence level (Manakul et al., 2023) and sub-sentence level checking (Min et al., 2023; Chern et al., 2023) have been proposed. However, these approaches still face limitations, as discussed in Sec. 1. In this paper, we employ knowledge triplets extracted from responses as claims which provide a structured framework for defining claim granularity.

**Hallucination Checking** One line of work for hallucination checking focus on resource-free checking with no requirements of reference. They mainly depend on self-contradiction (Mündler et al., 2023) or uncertainty (Zhang et al., 2023b) of the LLMs, or self-consistency between randomly sampled responses (Manakul et al., 2023; Chen and Mueller, 2023; Zhang et al., 2023a). The effectiveness of these methods depends on the LLM-based checker’s capability including knowledge coverage, instruction following ability and calibration. Furthermore, the self-consistency based methods needs to sample several responses for cross checking, which is costly. REFCHECKER is aligned with another line of work which requires references to check with (Min et al., 2023; Chern et al., 2023). In addition, REFCHECKER adopts a 3-way classification framework to cover unverifiable claims as opposed to the binary classification used in previous work, which can only distinguish factual and non-factual claims. Moreover, we have introduced

open-source solutions for both claim extraction and verification, diverging from the predominant reliance on proprietary LLMs in prior research.

**Hallucination Detection Benchmarks** The existing benchmarks for hallucination detection primarily focus on response-level detection (Lin et al., 2022; Yang et al., 2023), or limited to specific domains and tasks (Manakul et al., 2023; Min et al., 2023), or solely address factuality hallucinations (Chen et al., 2023; Chern et al., 2023; Wang et al., 2023). In contrast, our proposed benchmark offers a broader scope, encompassing a diverse range of tasks and domains. Moreover, our human evaluation process involves a more extensive examination of various LLMs with more responses.

### 3 REFCHECKER: Definition and Benchmark

Hallucinations are claims made by LLMs not supported by factual knowledge, which we refer to as references; detecting hallucinations involves comparing the claims against the references. This process depends on context settings, granularity of checking and how we categorize the hallucinations. We will discuss them in turn.

#### 3.1 Context Settings and Benchmarks

We differentiate three context settings covering various tasks and employ different benchmarks for each setting.

**Zero Context (ZC)** Tasks in this setting require the LLM to respond solely based on its internal knowledge. Therefore, in principle, references should be in the training corpus. In practise, we use NaturalQuestions (NQ) (Kwiatkowski et al., 2019) as proxy dataset to represent such stable knowledge. The knowledge seeking questions from NQ